

Solution Globale du TP : API Gateway Microservices (Sécurisée)

Partie 1 : Présentation et Architecture Unique du Point d'Entrée

L'**API Gateway** est configuré sur le port 3000. Il agit comme l'unique interface publique pour le client (Postman). Les requêtes reçues sont inspectées puis redirigées dynamiquement vers les microservices concernés via axios.

Pour maintenir la cohérence de la sécurité de l'architecture globale, le Gateway embarque désormais la clé secrète inter-services lors de la redirection des requêtes vers les services internes.

Partie 2 : Configuration du dossier "gateway"

1. Commandes d'installation

```
mkdir gateway
cd gateway
npm init -y
npm install express axios
npm install --save-dev nodemon
```

2. Fichier package.json

```
{
  "name": "api-gateway",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "nodemon index.js"
  },
  "dependencies": {
    "axios": "^1.6.0",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

3. Code complet du fichier index.js (Avec Sécurisation et Surlignage)

```
const express = require("express");
const axios = require("axios");

const app = express();
app.use(express.json());

// Définition des URLs cibles des microservices internes
const LIVRE_SERVICE = 'http://127.0.0.1:3001';
const MEMBRE_SERVICE = 'http://127.0.0.1:3002';
const EMPRUNT_SERVICE = 'http://127.0.0.1:3003';
```

```

// =====
// CONFIGURATION DE SÉCURITÉ INTER-SERVICES
// Le Gateway s'authentifie auprès des microservices avec la clé secrète partagée
// =====
const securityConfig = {
  headers: { 'x-api-key': 'mon_secret_microservice_123' }
};
// =====

// -----
// ROUTAGE : SERVICE LIVRE (Ports publics -> Service 3001)
// -----

app.get('/livres', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres`, securityConfig);

    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.get('/livres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres/${req.params.id}`,
    securityConfig);

    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.post('/livres', async (req, res) => {
  try {
    const r = await axios.post(`${LIVRE_SERVICE}/livres`, req.body,
    securityConfig);

    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.put('/livres/:id', async (req, res) => {
  try {

```

```

        const r = await axios.put(`${LIVRE_SERVICE}/livres/${req.params.id}`, req.body,
securityConfig);

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.patch('/livres/:id/disponibilite', async (req, res) => {
    try {
        const r = await axios.patch(`${LIVRE_SERVICE}/livres/${req.params.id}/
disponibilite`, req.body, securityConfig);

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.delete('/livres/:id', async (req, res) => {
    try {
        const r = await axios.delete(`${LIVRE_SERVICE}/livres/${req.params.id}`,
securityConfig);

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

// -----
// ROUTAGE : SERVICE MEMBRE (Ports publics -> Service 3002)
// -----

app.get('/membres/:id', async (req, res) => {
    try {
        const r = await axios.get(`${MEMBRE_SERVICE}/membres/${req.params.id}`,
securityConfig);

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.post('/membres', async (req, res) => {
    try {

```

```

        const r = await axios.post(`${MEMBRE_SERVICE}/membres`, req.body,
securityConfig);

        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

// -----
// ROUTAGE : SERVICE EMPRUNT (Ports publics -> Service 3003)
// -----

app.post('/emprunts', async (req, res) => {
    try {
        const r = await axios.post(`${EMPRUNT_SERVICE}/emprunts`, req.body);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.patch('/emprunts/:id/retour', async (req, res) => {
    try {
        const r = await axios.patch(`${EMPRUNT_SERVICE}/emprunts/${req.params.id}/retour`,
req.body);
        res.status(r.status).json(r.data);
    } catch (err) {
        if (err.response) return res.status(err.response.status).json(err.response.data);
        res.status(500).json({ message: err.message });
    }
});

app.listen(3000, () => {
    console.log('Gateway démarrée et sécurisée sur le port 3000');
});

```

Partie 3 : Guide de déploiement et de test global

1. Lancement de l'écosystème

Pour tester l'infrastructure de manière complète, ouvrez 4 terminaux séparés :

- **Terminal 1 (Port 3001):** `cd livre-service && npm run start`
- **Terminal 2 (Port 3002):** `cd membre-service && npm run start`
- **Terminal 3 (Port 3003):** `cd emprunt-service && npm run start`
- **Terminal 4 (Port 3000 - Passerelle):** `cd gateway && npm run start`

2. Validation sous Postman

Désormais, **toutes vos requêtes Postman ciblent uniquement le port 3000**. L'adresse change pour devenir :

- GET `http://127.0.0.1:3000/livres` (Redirigé vers le service Livre en s'authentifiant)
- POST `http://127.0.0.1:3000/emprunts` (Transféré directement au service Emprunt orchestrateur)